Youtube transcript of what TA said about the agentDev part of the lab

Okay, we're now inside the Jupyter lab. As you notice, there's some folders that may be missing uh that they are explained on the Redmi. The reason being that as you run the the agent and as you run sales on the Jupiter notebook, these folders will be created. So, no worries. They don't come when you fork, but they will be created. One of them is reports, the other one is logs and the other one is uh plots and also the work uh workplace so workspace. So don't worry they will be created as you work on the folder on the notebooks. So these are the three agentic uh notebooks. So agentic dev, agentic pipeline, agentic homework. So we're going to start with aentic dev. So, Gentic Dev is basically where we're going to be seeing uh how to build the tools and you're going to be building the tools. There's a summary of an entry uh explanation of what's an agent, what's a harness. We already saw it on the on the video, but it has more detail on the notebook. Also, as you notice, there's some bold uh words with a subscript. So, for example, large language model with a one that means that's one of the jargon or concepts from either agent or like computer science or maybe some syntaxes, but it means that it is on the glossaryy. So, you can go to the glossaryy and check what does it mean. So, where does the glossary mean lives? Glossy means it lives in agent dev. So glossaryy markdown uh you click it. Yeah. So this is the definition number one is large language model and you check the definition very straightforward. Okay. So going back I'm not going to be going through the whole notebook because I will be pretty lengthy. So I'm just going to do a review of the agent itself. But the notebook has several parts. It explains you that you got to be basically looking at three very important files. Tools, pipeline, breakdown, text, fixture, and backend XMA. The agent lives and dies with these three uh files. So, you got to read them. You got to know what's in them and you also need to give it to the agent. So, that's a very important detail. uh make sure you you read all the information on the notebook to understand what's happening what's the cycle it's very important okay it also explains you what's a testing fish a fixture explain you what's an eczema why it may actually cause problems or not it's very important you go through this so let's start with the lab so right now we got to put our keys right the keys we put inside the AMP. So yeah, the making the tools do consume a lot of rate. Uh so I recommend using all the keys. So one Grock, nine Gemini. If you happen to have a paid version, you can also use just one. Uh but for the ones who are using free tier, the more the better. So we run the the cell. The cell going to ask me for my name and my student ID. Okay. So, I'm going to put my name. Make sure to put your full name. I'm not going to be putting my full name right now, but make sure to put your full name because this is what it what that notebook uses to save your logs, save your plots, save your report name. So, if you if this doesn't match with your actual one, the grain is going to be complicated. So, the same with the student ID. So you put your student ID. Uh make sure it's always the correct one. Okay. And so I'm going to select you can select either Grog or Gemini. Let's do Grog. Important. Click the enable multi- provider fallback because without this there's no shuffling of the keys. The thing is if one of your keys hits an error or hits a rate limit the harness which we saw what is the harness which droughts it will it will switch the key for you and it will keep going. So the experience for you as a user is kind of flawless. So you don't feel you need to change the keys manually. just happens, right? But you need to have enable multi provider fallback and you need to have several keys. Okay, so I'm going to confirm it. Perfect.

So you keep reading the here. There's some examples of uh what you can prompt, some examples of how the interaction is going to be going and again some more examples. But remember that on the grading we grade the quality of your prompts. We grade that your prompts are original meaning that you are not copying directly these prompts. So if you copy these prompts you're graded less. Uh we're we're also grading the quality of the prompts. So that means like are you actually interacting with the agent or you just like yes do this fix that that's it also points. So you got to be able to talk with the agent and understand what the agent is doing so you own the project. Okay. So now we go to this uh this cell and this cell is where we will going to be engaging the agent. So this is the chat box of the agent. So we're going to start with interacting with the agent. So this is exactly where we build the tools. Remember agent dev notebook is where we're going to be building the tools. The tools is what the agent does or that the agent use to do things. So in this case, we're going to build tools for the agent to be able to do the lab with us. Okay, we're going to guide the agent right now on doing the tools. So the first of all, uh I need I name the tool and the point at the contract and then I don't need to describe the tool myself from memory. Why? because there's a file that already contains what the tool is and what we expect from the tool. This is called a specd driven development which is really important in authentic AI. We're not starting with a zero idea, right? The goal here is you do a objective and description of what you want the agent to build. Then you give this to the agent so the agent can absorb it and then propose to you maybe a better idea or some switches or maybe it will be like okay it's really good and you interact with the agent and that's how you cooperate with the agent instead of just I need a tool to do this okay no you took some time to understand what you need and based on your needs the agent can perform better so let's start by saying what do we need to the agent and we are going to make the agent go to tools pipeline breakdown. So what's tools pip pipeline breakdown? It's here, right? Uh we open it. It's very important you f familiarize yourself with it. It's a list of all the the tools. So this is a a markdown. So it may not look attractive here, but if you read the markdown on your GitHub, it's going to render much better because it actually has a markdown render. It doesn't have here. So right now it just appears like a simple text file. So it looks kind of clunky. But if you read it on the GitHub, it will look much cleaner and better. So when you read it, read it on GitHub. So here it tells what are we building, the tools we're building, and it also tells the contracts, the purpose for each tool. Right? So this is very important because you're going to be giving this to the agent. the agent reads it and that's where the agent knows what what it's building, right? You also need to know what is in here. You don't need to memorize it, but you need to have an idea because you're going to be interacting with the agent based on this file. Okay? So, we go back and we're going to be telling the agent that we are going to build a tool and we're gonna name the tool. Right? So, this is where we're telling the agent. Okay? So we're telling the agent to I want to build or let's I want to start with check missing tool. This is the tool we're going to be build, right? So open tools pipeline breakdown uh markdown where we were right now and tell me what is what its row says about this tool. Okay. So basically what we are saying is like read the file, familiarize yourself with it and tell me what's going on. We we we prompt you see uh Grog uh hit rate limit because I've been doing test. So that means that Grog has uh no more rates. So it's not working for me. But as you see the experience is smooth for me. It just tells me it hit rate limit. It got removed from the list but nothing more. It keeps going. Okay. So now the agent respond to me. It tells me where it's going to be saved, the tool name, and what does it does.

There's a slash command where you can check how many how many keys you have left with this slash status. And here tells me I'm using key one from Gemini and I already have one uh key on red limit. So by doing this I can quickly know how many more keys I have left. So this is a very useful way for you to let's say an initiate your keys. Okay. So it tell me the file uh to me uh tell me the tool name what it does. So I read it. I digest what it's saying. And now I want to say okay uh the spec leaves some room for interpretation. So I will pin it down before any code exists. Right? So I'm going to I'm going to prompt it again. I'm going to do the following. Okay. The contract only only says missing or empty text. in your plan will a string that's only spaces count as empty and a nan which is also an empty that comes in from from the CS loader I want both cut so we don't we want to leave any empty because there's several ways for empty to be interpreted. So it's like none no uh no string or a empty string because an empty string is technically empty in the semantic meaning but it's not technically empty if you judge it as non value or new value right uh so I want both to be caught so tell me how you do handle each one so we prompted again and Let's see what it responds. So now it responds with uh you know how it's going to handle. Uh so this is me interacting with the agent, not only just being like yes yes yes. So I'm understanding what is happening. I'm responding. This is a proper interaction with the agent. So the agent is is is is proposing to me uh what the steps can be done uh how it's going to be handled and how it's going to be insured. So if I agree with what the agent is saying I go and okay I can either be like I understand and I like what you're saying say it is logic or uh not right. So since it's it's talking about counts, let's do the following. Besides the count, should it also return which rows are affected? I do find the row position useful later when I decide whatever to drop them to drop them. Okay. Does the contract pin down the output or is that up to us? So we're now asking about the the contract again. Is it good? Is it bad? Okay. So agent response you see very seamless uh experience. Uh it goes back to read the the pipe the tool pipeline breakdown and it responds to to our questions. Right. So uh I mean uh it is suggesting to to check the test [sighs] fixture before to see if there's any inspect uh expected no answers there. Uh but the text fixture is a deterministic way to understand if the tool works or not. So we haven't talked about that file but this is the file the text fixture the same it's uh if you go into GitHub and you read it as a markdown it's going to be much better uh than reading it here from Jupiter lab but basic basically the test fixture tells what does each tool needs to to check right so the text fixture here is pretty plain so that means that uh it's missing in one row. So yeah, I mean why not? Let's let's say to agent just uh go ahead and read uh check the file. So literally the agent just read what we read. So now it's proposing how it the test uh should be done right. So let's let's go back to because now the agent is dering a little bit. So let's go back to the tool writing the tool design. What do you propose? So based on so we're going to wait for agent to do the final proposal now that it has all the context. Okay. So uh it's arrays just enough to use the tool add tool which is good for the for the form of the API. So we also want the agent to doublech check the XEMA. So let's say double check the back end sema notes MD MD and make sure uh it complies with the if. So, we're going to make sure the XEM is properly because we don't want any errors down the line. Okay. Uh so good. You see uh it checked the Excima. So, it's what it needs for Gemini and Gro. So, it doesn't uh crash. It comp it it is correct. It works. So we already check uh the three main I mean the the three files we need the agent to check which is the uh tools pipeline to understand what the agent is doing. We interact we ask questions about it. Uh we try to see if everything's covered. Then we saw the the text fixture to

understand it a little bit more. We gave it the context to the agent and also we made sure the agent was aware of the eczema so that the tools don't break down. Okay. So, uh we don't needed to check the the workpace. So, let's say uh I agree with the design. Let's write the tool. so the workspace is empty. Good. Because we're building the first tool. Yeah, we wanted to create tools data.pi because that's where the tools of this one is going to be living. Um so all right. So since the agent it's only allowed to do one test at a time uh it proposed tools data.pi containing check uh missing tools. I told you to write it but it can only propose or write in the same like turn. That's why it kind of repeats itself. But that's a design that I chose for because we are trying to avoid someone telling the agent, hey, just propose, right, and finish everything in one round. So yeah, so the agent is one of the assumptions is we'll only perform one test per round. So we we review the code. uh basically he's proposing this. So uh the whole code for it it should be okay because we are we we have been given it the directions from it. So this is just like the code representation of what we have already saw. So uh let's say I approve. So now it's been reading to the workplace. So let's go check if it's actually there. So the new folder it's been created. It's called agent works workplace. We click it and tools data is living there. That's not the tool itself. That's a file where the tool lives the script where the tool lives. So yeah it works. So now that we have the tool uh okay so we want to test for it. So let's build the text for the tool. Base it on. Let's check the name of file. Base it on uh text fix picture MD. We already read it and we saw it just saying one line, but we're reingesting it so the agent keeps remembering it. It's going to propose us uh how to build it. So, yep. It Yeah. So, it's asserting that the missing is one. Uh the has a missing is true and the total is seven and eight. So, yep. It it it works. Uh yes. Please uh the missing line test sins completely. This missing test of this tool itself is pretty simple. So there's not a lot to check but some other tools are a little bit more complex. So you should dig more on if it's really hitting all the all the parts of the test because as we saw this test was uh was pretty simple. It's just exactly one line. So since I'm aware of it that is quite simple then I'm being able to yeah it does cover it. But if it was an tool where there's more things to test we got to understand better the fixture to be able to improve it. Okay. So it propose uh let's say I have proposed okay uh yeah okay let's approve and it's going to be writing the test as we saw in the first uh round prompting now Gemini key1 hit a red limit so it was removed from the from the But it's okay because we as users don't have to do anything. Uh we're just here now. It tells me there are eight of them still alive. Okay. So that's why we say manage your rate limits properly and start the live with time because it does consume a lot of rate limits. So the test tools has been created and it's asking me to run the test. So yes, please run that test. So now the agent was going to run the test with the keys we just created. So the test pass [snorts] it cleanly. Check missing tool correctly identifies row seven that is missing. Uh now it's asking me if I want to continue building more tools. So congratulations you built your first tool. As I mentioned the tool is in agent dev workspace. Uh the tool is here and inside the test is the test. Okay. So uh if I want to check again how is my tools sorry my keys slash status and yep it tells me I have two rate limit tools so I got to be careful with this if I'm going to continue working in this section in this uh session I got to be mindful of this another thing is in this notebook you can pause and come back another day to work because as you uh as we're building tools uh each tool doesn't depend on the other one for building them. So I'm completely fine to stop working here because uh the scripts are saved in agent dev workspace. So there's like I I could come back tomorrow, restart the kernel and start working from the next tool and there's no problem. But when we work

on a gentic uh pipeline and homework, we cannot stop because the memory inside the exercise that is being built is completely wipe if we restart or stop. So we got to build it and we got to run it with really fresh rate uh keys and in one sitting. So please book proper time to do it and be mindful of your rate limits. So okay now that we built one as an example we can move on. Uh now for submitting your work uh the session is already creating the log. You don't really need to do anything to create the log. The log starts going. What's the log? The log is an encrypt file that keeps track of all your interactions with the agent. So we are going to be grading this and this lives in uh session logs which is right here. This uh file was created that's why it says 17 minutes ago because the first time we prompt the chat box it creates it and this uh says agent dev session. Okay. And you ask, so why it's not my name there? Because you remember that when we fill this the name student ID this meta data is being record already here. So you got to be really careful putting your correct name and your correct ID because this is already in embed inside this. Okay. So now final step uh you we just want to check that things are here. So we run this and yeah it just basically tell us that your log is there. So this uh zoom in your work it just uh check that you do that tells you yes your folder exists. So now we built one tool this tool and you're supposed to build all the tools here right all the tools you're going to be using because in the next notebooks in pipeline and homework you're going to be using these tools. So now I'm going to explain how to work on these ones.